

Axis vs. PyTorch: 7B-Parameter LoRA Fine-Tuning Performance

A controlled single-GPU comparison of training throughput and memory footprint

DATE	ACCELERATOR	VERSION	CLASSIFICATION
24 July 2026	NVIDIA A100-80GB	Axis 0.15.1	Public

§ Executive summary

On a single NVIDIA A100-80GB, **Axis** performs low-rank (LoRA) fine-tuning of a 7-billion-parameter transformer in **17% less GPU memory** than the reference PyTorch stack (Hugging Face Transformers + PEFT), while PyTorch retains a **~15% throughput advantage**. Both frameworks train an identical set of ~40M adapter parameters under matched precision and attention settings.

17% Lower peak GPU memory (Axis vs. PyTorch — 23.3 GB vs. 28.2 GB)	0.87× Relative training throughput of Axis (3,833 vs. 4,398 tok/s)	~40M Adapter parameters trained — identical for both frameworks
--	--	---

1 Objective

This report quantifies the training-time throughput and peak device-memory consumption of the **Axis** compiled CUDA engine relative to the mainstream PyTorch fine-tuning stack, at a representative 7B model scale. Full-parameter 7B training does not fit within a single 80 GB accelerator for either framework (bf16 weights + gradients + fp32 Adam state $\approx$ 112 GB); the parameter-efficient LoRA regime is therefore the appropriate single-GPU workload and is the subject of this comparison.

2 System under test

HARDWARE

Accelerator	NVIDIA A100-80GB (Ampere, SXM)
Provider	Modal — one dedicated container per configuration
Isolation	Fresh container per run (cold GPU state, no cross-run caching)

SOFTWARE

CUDA toolkit	12.4.1 (devel image, Ubuntu 22.04)
Python	3.11
Axis	0.15.1 — compiled C++/CUDA engine (own cuBLAS handle, fused flash attention)
Reference stack	PyTorch + Hugging Face Transformers (SDPA/flash) + PEFT (current release)

3 Model and workload

Architecture	Llama-2-7B shape (decoder-only, RoPE, RMSNorm, SwiGLU)
Hidden size / layers	4096 / 32
Attention heads	32 query, 32 key-value (MHA), head dim 128
MLP hidden	11,008
Vocabulary	32,000 (tied embeddings)
Adaptation	LoRA, rank 16, α 32, targets q/k/v/o/gate/up/down (~40M trainable)
Sequence length	2,048
Batch size	1
Precision	bf16 compute, fp32 optimizer state
Optimizer	AdamW (lr 3e-4, β 0.9/0.95, weight decay 0.1)

4 Methodology

- Matched configuration.** Both frameworks train the identical LoRA adapter set (same rank and target projections), the same model shape, and the same bf16 + flash-attention path.

- **Measurement.** 2 warm-up steps precede 5 timed steps; wall-clock time is captured with device synchronisation around the timed region. Reported latency is the per-step mean.
- **Throughput.** Tokens per second = (batch × sequence length) ÷ mean step time.
- **Memory.** Peak device memory is read from the driver (`nvidia-smi` , `memory-used`) and reflects the full process footprint.
- **Isolation.** Each configuration executes in its own fresh container to eliminate allocator and cache carry-over.

5 Results

FRAMEWORK	LATENCY (MS/STEP)	THROUGHPUT (TOK/S)	PEAK MEMORY	TRAINABLE
PyTorch (HF Transformers + PEFT)	466	4,398	28.2 GB	40M
Axis	534	3,833	23.3 GB	40M

Table 1. Single-A100 7B LoRA fine-tuning. Lower latency and memory are better; higher throughput is better.



6 Analysis

Memory. Axis's advantage stems from the parameter-efficient design of its compiled engine: the frozen 7B base carries no optimizer state and is held in bf16, so only the ~40M adapters incur fp32 master/moment storage. This yields a 4.9 GB (17%) reduction in peak footprint, translating directly into headroom for longer context windows or larger micro-batches on the same accelerator.

Throughput. PyTorch remains ~15% faster per step, reflecting a mature stack of hand-tuned CUDA kernels and deep PEFT integration. Axis closes much of this gap through CUDA-graph replay and fused flash attention but does not yet lead on raw LoRA throughput at this scale.

Consistency. The result is directionally consistent with the corresponding 1B-scale measurement (Axis ~38% lower LoRA memory, PyTorch marginally faster), indicating the memory advantage is structural rather

than scale-specific.

7 Scope and limitations

- Results characterise **LoRA fine-tuning** only. Full-parameter 7B training exceeds a single 80 GB accelerator for both frameworks and requires multi-GPU sharding (ZeRO / FSDP), evaluated separately.
- Measurements use a fixed sequence length (2,048) and batch size (1); throughput scales with both and should be re-measured for other regimes.
- Reference-stack library versions track the current public release at the report date and are not pinned.

8 Reproducibility

The benchmark harness and this report are published in the Axis repository. To reproduce on an A100-80GB:

```
modal run engine/modal_7b_bench.py
```

Source: github.com/Zyora-Dev/axis · Package: `pip install axis-zyora`